

PesaGuard

Real-Time Reconciliation & Anomaly Detection for M-Pesa-Integrated Businesses

Project Blueprint — Requirements, Architecture, Tools & Development Phases

Prepared for: Victor Kipruto Rop (DataForge)

Date: July 2026

1. Project Overview

PesaGuard is a real-time reconciliation and anomaly detection system for SACCOs, e-commerce operators, and small fintechs that accept payments via Safaricom's M-Pesa Daraja API. It closes the gap between what M-Pesa confirms was paid and what a business's internal system recorded — catching duplicate transactions, amount mismatches, missing callbacks, and till-number errors within seconds instead of days.

The Problem

Businesses using Daraja today typically reconcile payments manually in spreadsheets, often a day or more after the fact. In that window, money can go missing, customers can be double-charged, and failed webhook callbacks go unnoticed until a customer complains.

The Value Proposition

PesaGuard doesn't sell "a data pipeline" — it sells the guarantee that missing or mismatched money is caught before the customer notices. That reframes it from an infrastructure purchase to a risk-reduction purchase, which is a sharper, easier sell to a business owner or SACCO treasurer.

Target Customers

- SACCOs managing member contributions and loan repayments via M-Pesa
- E-commerce operators taking Till/Paybill payments at volume
- Small fintechs building on top of Daraja without in-house reconciliation tooling
- Chama-style community finance groups needing an audit trail

2. Requirements

2.1 Functional Requirements

ID	Requirement	Priority
FR-1	Receive and validate M-Pesa Daraja C2B confirmation/validation webhooks	Must
FR-2	Publish validated transaction events to a durable stream (Kafka)	Must
FR-3	Match incoming M-Pesa transactions against internal ledger/order records	Must
FR-4	Detect duplicate transaction IDs	Must
FR-5	Detect amount mismatches between M-Pesa and internal records	Must
FR-6	Detect missing callbacks (internal record exists, no matching payment)	Must
FR-7	Flag unusually large transactions for manual review	Should
FR-8	Send real-time alerts (Slack, SMS, email) on discrepancies	Must
FR-9	Provide a dashboard showing reconciliation status and history	Must
FR-10	Allow manual resolution/annotation of flagged discrepancies	Should
FR-11	Support multiple tenants (customers) with isolated data	Must

ID	Requirement	Priority
FR-12	Support pluggable internal-system connectors (Postgres, Sheets, REST API)	Must
FR-13	Daily summary digest of reconciliation activity	Could
FR-14	Auto-resolve discrepancies when a late-arriving event matches	Could

2.2 Non-Functional Requirements

ID	Requirement	Target
NFR-1	Reconciliation latency from webhook receipt to match/flag	< 30 seconds
NFR-2	No data loss on service downtime (buffering/DLQ)	Zero dropped events
NFR-3	Idempotent processing of duplicate webhook deliveries	100% dedup
NFR-4	System uptime for webhook receiver	99.5%+
NFR-5	Auditability — every decision traceable to a rule and timestamp	Full audit log
NFR-6	Tenant data isolation	No cross-tenant data leakage
NFR-7	Alert delivery latency	< 60 seconds from detection

3. Tools & Tech Stack

Layer	Tool / Technology	Purpose
Ingestion	Flask	Webhook receiver for M-Pesa Daraja callbacks
Streaming	Apache Kafka	Durable event stream between services
Processing	Python (MVP) / PyFlink (scale)	Reconciliation & anomaly detection logic
Storage	PostgreSQL	Transactions, discrepancies, tenant config, audit log
Storage (scale)	BigQuery	Analytics/reporting once volume justifies it
Connectors	psycopg2, gspread, requests	Postgres / Google Sheets / generic REST integrations
Alerting	Slack Webhooks, Africa's Talking (SMS)	Real-time discrepancy notifications
Dashboard frontend	Next.js + React	Live discrepancy feed, charts, tenant settings
Dashboard backend	Flask / Django REST Framework	API serving dashboard data
Auth	Email/password or magic link (e.g. NextAuth)	Dashboard access control
Containerization	Docker, docker-compose	Local dev and single-VM production deploy
Orchestration (scale)	Kubernetes	Only once supporting 5+ tenants justifies the ops cost
IaC (scale)	Terraform	Reproducible cloud infra once beyond single-VM
CI/CD	GitHub Actions	Test, build, deploy pipeline
Observability	Structured JSON logging, Prometheus-style /metrics	Latency, throughput, error tracking
Testing	pytest	Unit + integration tests

4. System Architecture

M-Pesa Daraja sends webhook callbacks to the Flask receiver, which validates the payload and publishes it to a Kafka topic. The reconciliation service consumes this stream, pulls matching records from the tenant's internal system via a pluggable connector, applies matching and anomaly rules, and writes results to Postgres. Discrepancies trigger alerts (Slack/SMS/email); the dashboard reads from Postgres to show live status.

Data Flow

- M-Pesa Daraja → Webhook Receiver (validate, dedupe by TransID)
- Webhook Receiver → Kafka topic `mpesa.transactions.raw`
- Reconciliation Job → consumes stream, queries internal-system connector
- Reconciliation Job → writes matched/discrepancy records to Postgres
- Discrepancy detected → Alerting service → Slack / SMS / email
- Dashboard API → reads Postgres → Next.js frontend renders live status

5. Development Phases

A phased build order, structured to get a working pilot in front of one real customer as early as possible rather than building the full platform first.

Phase	Focus	Timeline	Key Deliverables
Phase 0	Discovery & Validation	Week 1	Interview 5-10 SACCOS/e-commerce operators on current reconciliation pain. Confirm the problem is real and validate willingness to pay before writing code.
Phase 1	MVP Core Pipeline	Weeks 2-3	Webhook receiver, Kafka stream, basic anomaly rules (duplicates, invalid amounts), Postgres storage, Slack alerting. Runs via docker-compose. (This is the scaffold already built.)
Phase 2	Real Matching Logic	Weeks 4-5	Implement actual cross-system matching: Postgres connector wired to a real pilot customer, phone+amount+time-window matching, partial-match handling, missing-callback detection.
Phase 3	Pilot Deployment	Week 6	Deploy to a single VM, onboard one real paying/piloting customer, run in shadow mode (alert-only) for a week before trusting it operationally.
Phase 4	Dashboard & Manual Resolution	Weeks 7-8	Next.js dashboard: live discrepancy feed, reconciliation rate chart, manual resolution flow, tenant settings page.
Phase 5	Multi-Tenancy & Additional Connectors	Weeks 9-10	Tenant-scoped data model, Google Sheets and generic REST connectors, connector config UI so onboarding customer #2+ doesn't need a code change.
Phase 6	Alerting Maturity & Observability	Week 11	SMS alerts (Africa's Talking), severity tiers, escalation on unresolved critical alerts, daily digest, structured logging, health checks, metrics.
Phase 7	Hardening & Testing	Week 12	Full unit + integration test coverage of matching scenarios, load testing, dead-letter queue for failed processing, idempotency audit.
Phase 8	Launch & Iterate	Week 13+	Onboard additional pilot customers, gather resolution-note data to tune thresholds, evaluate PyFlink/Kubernetes migration only if volume justifies it.

6. Roles & Skills Needed

Role	Needed For	Notes
Backend / Data Engineer	Ingestion, streaming, reconciliation logic	Covered by existing skillset (Kafka, Python, Postgres)
Frontend Engineer	Dashboard (Next.js)	Can be self-taught or contracted for Phase 4
Domain / Ops advisor	Validating matching rules against real SACCO/fintech workflows	Early pilot customer can partly fill this role
DevOps (part-time)	CI/CD, deployment, monitoring setup	Minimal at MVP scale; single VM is enough initially

7. Risks & Assumptions

Key Risks

- **False positives erode trust** — if legitimate payments get flagged too often (e.g. family paying on someone's behalf), customers will start ignoring alerts. Mitigation: *needs_review* tier instead of hard discrepancy for ambiguous cases.
- **Daraja callback reliability** — Safaricom's webhook delivery isn't always immediate; matching windows must be tuned, not assumed, against real sandbox/production behavior.
- **Regulatory/compliance exposure** — handling financial transaction data for SACCOs may carry data protection obligations (Kenya's Data Protection Act) worth reviewing before onboarding real customer data.
- **Single point of failure in MVP** — Kafka/Postgres downtime during pilot must not silently drop transactions; dead-letter handling is not optional even at MVP scale.
- **Connector fragility** — every customer's internal system is different; the generic REST/Sheets connectors will need real-world hardening per integration.

Assumptions to Validate

- Pilot customers are willing to grant read access to internal order/ledger systems
- Daraja sandbox behavior is representative enough of production for early testing
- A Slack-first alerting flow fits the pilot customer's existing ops habits (vs. needing SMS from day one)

8. Success Metrics

Metric	Why It Matters
Time-to-detection (webhook receipt → flag)	Core value proposition: speed beats manual reconciliation
False positive rate on flagged discrepancies	Determines whether customers trust and act on alerts
% of transactions auto-reconciled without human review	Measures how much manual work is actually removed
Customer-reported "money recovered/caught" instances	The concrete story that sells the next customer
Pilot-to-paid conversion rate	Validates the business model, not just the technology

9. Immediate Next Steps

- Run Phase 0 customer interviews before writing more code
- Confirm Daraja sandbox access and test real callback payload shapes
- Wire the Postgres connector to one real (or simulated) pilot customer dataset
- Deploy the existing MVP scaffold to a single VM and run it in shadow mode
- Use the earlier full build prompt (PESAGUARD_BUILD_PROMPT.md) with an AI coding agent to accelerate Phases 2-7 once the pilot validates the core problem